LR1 — High-Availability System Architecture

Topic

Design of a high-availability system architecture using cloud technologies.

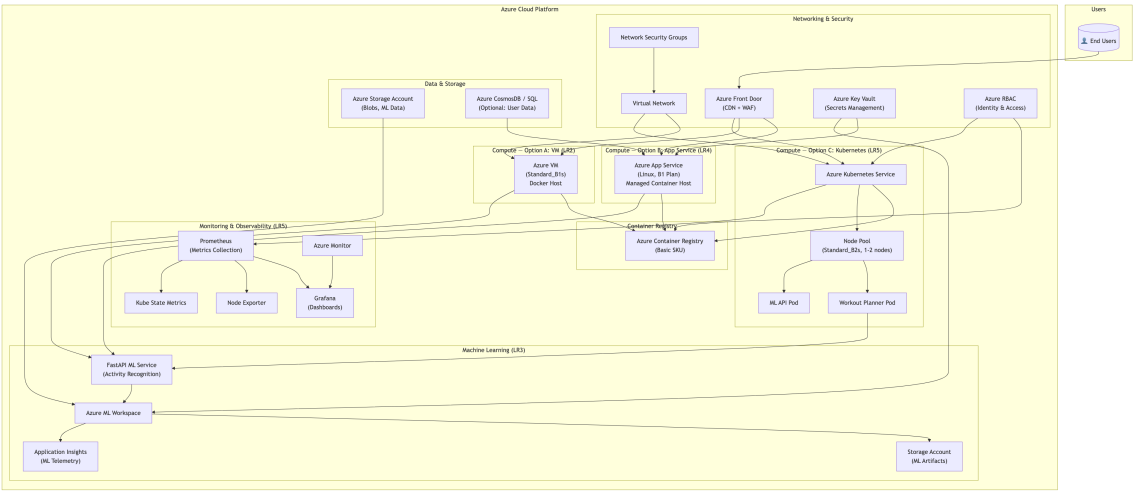
Objective

Develop the architecture of a continuously available system leveraging Azure cloud platform. Show the relationships between Azure services required for hosting the system in the cloud. Describe key recommendations for migrating an existing system to the cloud.

Target System

Workout Planner — a web application that provides personalized workout recommendations using machine learning, deployed on Azure with full monitoring and observability.

Architecture Diagram



Azure Services Overview

Service	Role	Lab
Azure Resource Group	Logical container for all resources	All
Azure Virtual Network (VNet)	Isolated network for VMs and services	LR2, LR5
Network Security Groups (NSG)	Firewall rules (HTTP/HTTPS/SSH)	LR2
Azure Container Registry (ACR)	Private Docker image registry	LR2, LR4, LR5
Azure Virtual Machine	Docker host for low-level container deployment	LR2

Azure App Service	Managed PaaS for containerized web apps	LR4
Azure Kubernetes Service (AKS)	Managed Kubernetes for orchestrated deployment	LR5
Azure ML Workspace	ML experiment tracking, model registry	LR3
Azure Storage Account	Blob storage for ML data and artifacts	LR3
Azure Key Vault	Secrets, certificates, encryption keys	LR3, LR4
Azure Application Insights	APM and telemetry for ML service	LR3
Azure Monitor	Cloud-native metrics and logs	LR5
Azure Front Door	Global load balancing, CDN, WAF	Architecture
Azure RBAC / Entra ID	Identity and access management	All

Quality Attributes

1. Availability

- **Azure App Service**: Built-in 99.95% SLA, automatic failover
- **AKS**: Multi-node pool with self-healing (pod restart policies, liveness/readiness probes)
- **Availability Zones**: Resources distributed across fault domains
- **Health checks**: `/health` endpoints on all services for monitoring and auto-recovery
- **Auto-scaling**: App Service scale-out rules; AKS Horizontal Pod Autoscaler (HPA)

2. Scalability

- **Horizontal scaling**: AKS HPA scales pods based on CPU/memory; App Service auto-scale rules
- **Vertical scaling**: VM size can be upgraded; App Service plan tier can be changed
- **Container Registry**: Serves images to any number of compute targets
- **Stateless design**: App containers are stateless; state stored in external storage

3. Security

- **Network isolation**: VNet + NSG rules restrict traffic to known ports
- **Secrets management**: Azure Key Vault stores all credentials, connection strings, API keys
- **Managed identities**: App Service and AKS use managed identities to access ACR and Key Vault (no stored passwords)
- **RBAC**: Fine-grained access control via Azure Entra ID roles
- **TLS/HTTPS**: App Service provides managed SSL certificates; Front Door terminates TLS
- **Container security**: ACR supports image scanning and signing

4. Data Storage

- **Azure Storage Account**: Blob storage for ML training data, model artifacts, and logs
- **CosmosDB/SQL** (optional): Persistent user data, workout history
- **Container Registry**: Immutable image layers stored securely

5. Monitoring

- **Prometheus**: Collects metrics from all K8s workloads via node-exporter and kube-state-metrics
- **Grafana**: Visualization dashboards for CPU, memory, HTTP latency, error rates

- **Azure Monitor / Application Insights**: Cloud-native telemetry for App Service and ML workspace
- **Alerting**: Prometheus alert rules for CPU > 80%, pod restarts > 3, high error rates

---

Cloud Migration Recommendations

1. **Containerize first**: Package the application as a Docker image to ensure portability across any cloud compute target (VM, App Service, AKS).
2. **Use Infrastructure as Code**: Define all resources with Terraform for repeatability, version control, and disaster recovery.
3. **Start with managed services**: Begin with App Service (LR4) for simplicity, then migrate to AKS (LR5) when orchestration features are needed.
4. **Externalize configuration**: Use environment variables and Key Vault instead of hardcoded settings.
5. **Implement observability early**: Deploy Prometheus + Grafana from day one to monitor application health and resource usage.
6. **Design for statelessness**: Keep compute instances stateless; persist data in Azure Storage or databases.
7. **Minimize costs**: Use free/student tiers where available; destroy resources after demos; set up budget alerts.
8. **Automate deployments**: Use scripts for build/push/deploy; consider CI/CD pipelines (GitHub Actions) for production.

Conclusion

The Workout Planner architecture demonstrates a modern cloud-native approach to application deployment. By using Azure services across VM (IaaS), App Service (PaaS), and AKS (CaaS) deployment models, the project covers the full spectrum of cloud computing paradigms. Terraform ensures infrastructure is reproducible and version-controlled, while Prometheus + Grafana provide comprehensive observability. The architecture supports high availability through managed services with built-in SLAs, horizontal scaling through AKS and App Service auto-scale, and security through VNet isolation, managed identities, and Key Vault.